

- I) Em uma arquitetura *encoder-decoder* usando redes recorrentes, qual a motivação de utilizar o mecanismo de atenção entre o *encoder* e o *decoder*?
- II) A figura abaixo ilustra o processo de atenção em uma rede recorrente. Nessa rede recorrente, temos que tanto os estados ocultos do *encoder*  $h_i$  quanto o estado oculto do *decoder*  $s$  possuem tamanho 2. Qual é o valor do vetor de contexto ao computar a atenção por produto escalar do estado do *decoder*  $s_1$  para todos os estados do *encoder*  $h_i$ ?

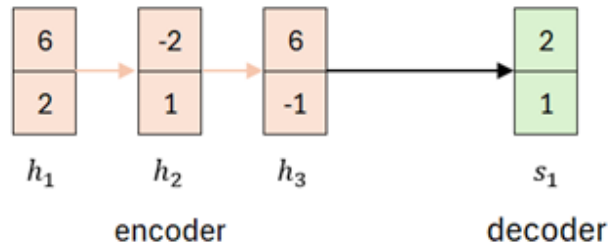


Figura 1: Diagrama contemplando os estados ocultos  $h_i$  de um *encoder* baseado em RNNs e o estado oculto de um *decoder*  $s_1$ . A representação dos tokens e dos *embeddings* dos tokens foram removidas da figura para simplificar o diagrama.

- III) (Prince, 2024) Considerando que um mecanismo de *self-attention* processa  $N$  entradas de tamanho  $D$  para produzir  $N$  saídas de tamanho  $D$ . Quantos parâmetros treináveis são usados para computar *queries*, *keys* e *values*? Caso seja utilizada uma camada linear densa ao invés do mecanismo de atenção, o número de parâmetros treináveis seria maior ou menor?
- IV) Qual é o limite teórico do tamanho de sequências que podemos processar com o mecanismo de *self-attention*?
- V) É comum no treinamento e na inferência de *transformers* limitarmos o tamanho das sequências tanto na entrada quanto na saída. Qual o motivo disso?
- VI) Por que a camada de *self-attention* no *encoder* do *transformer* original não possui máscaras e no *decoder* possui máscaras?
- VII) Qual a diferença entre *cross-attention* e *self-attention*?
- VIII) Qual a vantagem de utilizarmos *multi-head self-attention* em comparação a *self-attention*?

**Gabarito**

- I) Ao usar o mecanismo de atenção em uma arquitetura *encoder-decoder* estamos evitando (ou pelo menos minimizando) um gargalo de comunicação entre o *encoder* e o *decoder*. Isso ocorre pois sem a atenção, o *decoder* consegue ter acesso apenas a um estado oculto do *encoder* que deve ser capaz de representar toda a sequência armazenada.
- II)  $\begin{bmatrix} 6 \\ 1.86 \end{bmatrix}$
- III) Cada uma das três componentes da atenção, *query*, *key* e *values* demanda  $D \times D$  parâmetros treináveis mais  $D$  bias, sendo o total  $3D^2 + 3D$  para computar todo mecanismo de self-attention. Para fins de comparação, uma camada densa totalmente conectada demandaria  $(DN)^2 + DN$  parâmetros treináveis. Dessa forma percebemos que o número de parâmetros treináveis é menor com o mecanismo de *self-attention*.
- IV) Não há limite teórico para o tamanho da sequência no processamento por *self-attention*. Contudo sabemos que existe um custo computacional quadrático em relação ao tamanho da sequência, isso acaba criando limites práticos para o tamanho de sequências que conseguimos processar. (Podemos ser preciosistas ao responder essa questão e apontar que apesar de conseguir computar a atenção em uma sequência de tamanho arbitrariamente grande, o limite efetivo que conseguimos processar tem relação com o tamanho dos vetores de *query* e *keys*, pois queremos ser capazes de diferenciar a similaridade entre todos os vetores processados).
- V) Existem vários motivos para limitarmos o tamanho da sequência tanto em treino quanto em inferência sendo que praticamente todos remetem a questões de eficiência. Por exemplo, temos que garantir que exista vram suficiente para o treinamento e uma forma de fazer isso é limitando o tamanho das sequências; outra questão importante é garantirmos que o modelo consegue interpretar adequadamente os *positional embeddings*, embora em teoria o modelo deva ser capaz de generalizar para sequências maiores na prática isso pode não ocorrer; Durante o treinamento queremos ser capazes de colocar todas as sentenças de um *batch* em um único tensor, para permitir esse processo é comum completarmos sequências mais curtas com <PAD> para facilitar esse processo limitamos o tamanho das sequências e também implementamos um processo de *bucketing*.
- VI) Durante o *treino* com *teacher forcing*, o *decoder* recebe toda a sequência alvo de uma só vez, em paralelo. A máscara causal impede que a posição  $t$  veja as posições futuras  $t + 1, t + 2, \dots$  ao computar a atenção, garantindo que cada predição use apenas o contexto que estará disponível em inferência (autoregressiva). Sem a máscara, o modelo simplesmente copiaria o próximo *token* da sequência alvo (trapaceando) e não aprenderia a prever. O *encoder*, ao contrário, processa a frase de entrada de uma só vez e toda ela está disponível ao mesmo tempo, então não há razão para mascarar.
- VII) Em *Cross-attention* as *querys* e *keys* vem de sequências diferentes enquanto que em *self-attention* as *querys* e *keys* vem da mesma sequência.
- VIII) A principal vantagem é representacional: cada cabeça aprende a atender a um **tipo de relação diferente** entre os *tokens* da sequência, por exemplo, relações sintáticas (sujeito-verbo, modificador-substantivo), semânticas (referente de um pronome), posicionais (próximo *token*) ou de longo alcance. Trabalhos como Voita et al. (2019) *Analyzing Multi-Head Self-Attention* mostram empiricamente que cabeças individuais especializam-se em diferentes funções linguísticas. Como benefício secundário, o cômputo das múltiplas cabeças é trivialmente paralelizável

(operação *batched* em GPU), mas esse não é o motivo principal, uma única cabeça de mesma dimensão total seria igualmente paralelizável.